

Lecture 7: Core LeetCode Problem Solving Session (Integer & Bitwise Manipulation)

Problem 1: LeetCode 7 - Reverse Integer

1. Problem Statement & Constraints: Given a signed 32-bit integer X , return X with its digits reversed. If reversing X causes the value to go outside the signed 32-bit integer range $[-2^{31}, 2^{31} - 1]$, return 0.

2. Core Intuition: Reversing a number relies on standard base-10 digit extraction. We can extract the rightmost digit using modulo 10 ($x \% 10$) and append it to our answer by multiplying the current answer by 10 and adding the extracted digit ($ans = ans * 10 + digit$).

- **The Overflow Trap:** The critical constraint is the 32-bit environment. If our running accumulator (ans) is already greater than $INT_MAX / 10$, the very next step ($ans * 10$) will instantly cause a memory overflow. We must evaluate the overflow condition *before* we execute the multiplication.

3. Algorithmic Steps: 1. Initialize a running accumulator $ans = 0$.

2. Open a `while` loop that continues as long as $x \neq 0$.

3. Extract the last digit: $int\ rem = x \% 10$.

4. Evaluate the boundary safety check: If $ans > INT_MAX/10$ OR $ans < INT_MIN/10$, immediately return 0 (Overflow detected).

5. Update the accumulator: $ans = (ans * 10) + rem$.

6. Discard the processed digit from the original number: $x = x / 10$.

7. Return ans when the loop terminates.

4. Dry Run:

Input: $X = 123$

1. $rem = 123 \% 10 = 3$. Boundary check passes. $ans = 0 * 10 + 3 = 3$. $X = 12$.
2. $rem = 12 \% 10 = 2$. Boundary check passes. $ans = 3 * 10 + 2 = 32$. $X = 1$.
3. $rem = 1 \% 10 = 1$. Boundary check passes. $ans = 32 * 10 + 1 = 321$. $X = 0$.
4. Loop breaks. Output: 321.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\log_{10} X)$	The loop executes exactly once for every digit in the base-10 integer.
Space Complexity	$O(1)$	No auxiliary data structures are used; variables consume strictly constant space.

```

class Solution {
public:
    int reverse(int x) {
        int ans = 0;
        while (x != 0) {
            int rem = x % 10;

            // Critical Overflow Check prior to multiplication
            if (ans > INT_MAX/10 || ans < INT_MIN/10) {
                return 0;
            }

            ans = ans * 10 + rem;
            x /= 10;
        }
        return ans;
    }
};

```

Problem 2: LeetCode 1009 - Complement of Base 10 Integer

1. Problem Statement & Constraints: The complement of an integer is obtained by flipping all 0s to 1s and all 1s to 0s in its binary representation. Given an integer N , return its base-10 complement.

2. Core Intuition: Using the bitwise NOT operator ($\sim N$) successfully flips all bits. However, a 32-bit integer contains numerous leading zeros. The NOT operator will flip all of those leading zeros into leading ones, generating a massive, incorrect number.

- **The Masking Solution:** We must create a "Mask" consisting of consecutive 1s that is the exact same bit-length as N . By applying a bitwise AND ($\&$) between our inverted number ($\sim N$) and our `mask`, we effectively erase the infinite leading ones and isolate only the significant bits.

3. Algorithmic Steps: 1. Handle the strict edge case: If $N == 0$, immediately return 1.

2. Create a proxy variable `m = n` to destroy during mask creation, and initialize `mask = 0`.
3. Open a `while` loop bound by `m != 0`.
4. Update the mask by left-shifting it and appending a 1: `mask = (mask << 1) | 1`.
5. Right-shift the proxy variable: `m = m >> 1`.
6. Once the mask matches the bit-length, return `(~n) & mask`.

4. Dry Run:

Input: $N = 5$ (Binary: `...000101`)

1. $N = 5$. $m = 5$.
2. Iteration 1: `mask = (0 << 1) | 1 = 1` (`...001`). `m = 5 >> 1 = 2`.
3. Iteration 2: `mask = (1 << 1) | 1 = 3` (`...011`). `m = 2 >> 1 = 1`.
4. Iteration 3: `mask = (3 << 1) | 1 = 7` (`...111`). `m = 1 >> 1 = 0`. Loop terminates.
5. Invert N : `~5` $\rightarrow$ `...111010`.
6. Apply Mask: `...111010 & ...000111 = ...000010` (Decimal 2). Output: 2.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(\log_2 N)$	The loop shifts bit-by-bit until the magnitude of the number is exhausted.
Space Complexity	$O(1)$	Constant allocation for <code>mask</code> and <code>m</code> .

C++

```
class Solution {
public:
    int bitwiseComplement(int n) {
        int m = n;
        int mask = 0;

        // Base case trap
        if (n == 0) {
            return 1;
        }

        // Generate a mask of 1s matching the bit-length of N
        while (m != 0) {
            mask = (mask << 1) | 1;
            m = m >> 1;
        }

        // Erase flipped leading zeros using the mask
        return (~n) & mask;
    }
};
```

Problem 3: LeetCode 231 - Power of Two

1. Problem Statement & Constraints: Given an integer N , return `true` if it is a mathematical power of two ($N = 2^x$ for some integer x). Otherwise,

```
return false .
```

2. Core Intuition: A 32-bit signed integer has a strictly limited capacity. The absolute maximum power of two it can hold is 2^{30} . We can algorithmically generate every valid power of two from 2^0 up to 2^{30} and simply check if our target N matches any of them.

- **The Multiplication Overflow:** If we generate 2^{30} and multiply it by 2 to prepare for the next loop, it becomes 2^{31} , instantly causing an integer overflow. We must bound our generator multiplication with `INT_MAX / 2`.

3. Algorithmic Steps: 1. Initialize the lowest power of two: `ans = 1` (2^0).

2. Open a `for` loop executing exactly 31 times (`i = 0` to `30`).

3. Inside the loop, check if the current power of two equals N (`ans == n`). If True, return `true`.

4. Evaluate overflow guard: If `ans < INT_MAX / 2`, mathematically calculate the next power of two: `ans = ans * 2`.

5. If the loop exhausts all 31 possibilities without a match, return `false`.

4. Dry Run:

Input: $N = 16$

1. $i = 0$: `ans = 1`. $1 \neq 16$. `ans = 1 * 2 = 2`.
2. $i = 1$: `ans = 2`. $2 \neq 16$. `ans = 2 * 2 = 4`.
3. $i = 2$: `ans = 4`. $4 \neq 16$. `ans = 4 * 2 = 8`.
4. $i = 3$: `ans = 8`. $8 \neq 16$. `ans = 8 * 2 = 16`.
5. $i = 4$: `ans = 16`. $16 == 16$. Returns `true`.

5. Complexity Analysis Table:

Metric	Complexity	Justification
Time Complexity	$O(1)$	Regardless of input size, the algorithm runs a fixed maximum of 31 iterations.
Space Complexity	$O(1)$	Tracks a single scalar variable <code>ans</code> .

C++

```
class Solution {
public:
    bool isPowerOfTwo(int n) {
        int ans = 1;
```

```
// A signed 32-bit integer can only hold up to 2^30 safely
for (int i = 0; i <= 30; i++) {

    if (ans == n) {
        return true;
    }

    // Prevent multiplication overflow on the final iteration
    if (ans < INT_MAX / 2) {
        ans *= 2;
    }
}
return false;
};
```

(Note: This problem can also be solved optimally in $O(1)$ time using the bitwise trick `return n > 0 && (n & (n - 1)) == 0;`, but the generation strategy establishes fundamental overflow protection mechanics).